

Ödev: Bilet Alma Akışını Uçtan Uca Tamamla

TicketApp · Orta seviye · Android (Kotlin + Compose) · 2026-05-23

Hedef: Bir USER giriş yaptıktan sonra etkinliği seçip bilet alabilsin ve QR'ını görebilsin.

Tamamlanacak API akışı (referans: [API.MD](#)):

```
GET /events → GET /events/{id} → POST /purchases → POST /purchases/{id}/pay  
→ GET /me/tickets → GET /me/tickets/{id}
```

Mevcut mimariyi (3 modül: `app` / `core` / `data`, Koin, Retrofit, Result tabanlı repository) **bozmadan** ilerle. Her yeni endpoint için aynı kalıbı uygula:

DTO → API interface → Mapper → Domain model → Repository (interface + impl) → ViewModel → Screen.

1 Register Screen'i bitir

`AppNavHost.kt:64` şu an `Text("Register Screen")` placeholder.

- `RegisterScreen` + `RegisterViewModel` yaz, mevcut `LoginScreen.kt` ve `LoginViewModel.kt` ile bire bir paralel olsun.
- `AuthRepository.register` zaten hazır; sadece kullan.
- Validation:** email format + şifre 8–128 karakter (API kuralı).
- Hata mapping:** `409 email_taken` → "Bu email zaten kayıtlı".

2 Event Detay ekranı

- `core/domain/event/EventRepository` 'ye `getEvent(id: String): Result<Event>` ekle.
- `EventApi.kt` 'a `@GET("/events/{id}")` ekle, `EventRepositoryImpl` 'de kullan.
- Yeni route: `@Serializable data class EventDetail(val id: String)` (`AppDestinations.kt`).
- `HomeScreen` 'deki `EventCard` tıklanınca `EventDetail(event.id)` 'ya navigate et.
- Detay ekranı içeriği:** isim, açıklama, yer, tarih (mevcut `DateFormatter.kt` ile formatla), bilet türleri listesi (her satırda `name`, `remaining` / `capacity`, fiyat). Her satırda `+/-` ile adet seçimi (0–min(20, remaining)).
- Sayfa altında "Toplam: ₺X" + **"Satın Al"** butonu (`totalCents` 'ı kuruştan TL'ye çevirip göster).

3 Sat Alım + Ödeme akışı

Yeni katman: `core/domain/purchase/` ve `data/.../purchase/`.

- **Domain:** `Purchase` , `PurchaseItem` , `PurchaseStatus { PENDING, PAID }` , `Ticket` , `TicketStatus` .
- **DTO'lar:** `CreatePurchaseRequestDto` , `PurchaseItemRequestDto` , `PurchaseDto` , `PurchaseItemDto` , `TicketDto` .
- **PurchaseApi:**
 - `@POST("/purchases")` `createPurchase`
 - `@POST("/purchases/{id}/pay")` `pay`
 - `@GET("/purchases/{id}")` `getPurchase`
- **PurchaseRepository** (interface + impl + Koin'e `single` olarak ekle).
- **ViewModel akışı:**
 1. Detay ekranındaki seçimlerle `createPurchase` → `purchase.id` al → "Ödeme Onayı" diyalogu/ekranı.
 2. Onayla → `pay(id)` → başarıdaysa **Biletlerim**'e navigate et.
- **Hata yönetimi:** `409 capacity_exceeded` → "Stok yetersiz, yenile" (etkinliği refresh et). `409 already_paid` , `403 not_purchase_owner` için kullanıcıya anlaşılır mesaj. `LoginViewModel.kt` 'daki `toUserMessage` paternini ortak bir yere taşıyıp genişlet (ipucu: `core/util/ErrorMessage.kt`).

4 Biletlerim + QR ekranı

- **MeApi:** `@GET("/me/tickets")` ve `@GET("/me/tickets/{id}")` .
- **TicketRepository:** `getMyTickets()` , `getTicket(id)` .
- **MyTicketsScreen** — bilet kartları (etkinlik adı, tarih, bilet türü, durum). Boş state + error state + pull-to-refresh.
- **TicketDetailScreen** — QR ekranı. `qrCode` UUID'sini QR'a çevir.
 - `io.github.g0dkar:qrcode-kotlin` ya da `journeyapps:zxing-android-embedded` ekle. Sadece `qrCode` alanını payload olarak kullan (API.MD §5.4 notu).
 - Ekran açıkken **ekran parlaklığını maksimuma** çek (kapı görevlisi okuyabilsin).
- Home'a "Biletlerim" butonu veya bottom nav ekle.

5 Logout + 401 davranışı

- `HomeScreen` 'e top bar koy, çıkış butonu → `authRepository.logout()` . `AppNavHost` zaten `isLoggedIn` flow'unu izliyor, login'e otomatik düşer.
- Refresh başarısız olduğunda `TokenAuthenticator.kt` tokenları temizliyor olmalı; doğru. Değilse temizle ki kullanıcı login'e itilsin.

Teslim kriterleri

- ☐ Yeni etkinlik açılır → bilet türü + adet seçilir → satın alma → ödeme → biletlerim listesinde görünür → QR ekranı render olur.
- ☐ Stok azalan etkinlik refresh edilince `remaining` güncel gelir.
- ☐ Tüm yeni ekranlarda **Loading / Error / Empty / Content** dört state.
- ☐ Hiçbir Composable'da doğrudan `Api` / `Repository` çağrısı yok — her şey `ViewModel` üzerinden.
- ☐ Yeni eklenen tüm Koin bağımlılıkları `DataModule.kt` ve `AppModule.kt` 'a kayıtlı.
- ☐ Kullanıcıya görünen string literal'ler `strings.xml` 'e taşınmış.

Bonus (Eğer kolay geldiyse aşağıdakileri de yapmaya çalışabilirsin)

- **"Satın Alımlarım" ekranı** (`GET /me/purchases`) — PENDING olanları "Ödemeye devam et" ile `pay` 'e bağla.
- **STAFF rolü için CheckinScreen** (kamera + `POST /checkin/scan`).
- Ekran döndürmede / process death sonrası state korunsun (`SavedStateHandle`).